

Multithreading

Multithreading means running multiple threads within a single process at the same time to perform tasks concurrently.

1. Process → Independent running program
2. Thread → Smallest unit of execution inside a process
3. Multithreading → Multiple threads executing concurrently within a process

Several threads in one process share the same memory, which is what makes them fast and what makes them difficult.

A reading order:

Threads and their cost → Creating one → The lifecycle → The race condition
→ synchronized → volatile and atomics → Locks → Deadlock
→ Executors → Future → Concurrent collections → Virtual threads

1. Threads and what they cost

	Process	Thread
Memory	Its own	Shared with the other threads
Created by	The operating system, expensively	The JVM, more cheaply
A crash	Affects only itself	Can take the whole process down
Talking to another	Through pipes or sockets	Through shared fields

That last row is the whole subject. Two threads reading and writing the same field need no mechanism to reach each other, and equally nothing stops them doing it at the same moment.

Almost every concurrency bug is shared mutable state. A field that no thread writes, or a field only one thread can see, needs none of the machinery below.

2. Creating a thread

Three ways, and only the last is used much.

```
// 1. extend Thread
class Worker extends Thread {
    @Override
    public void run() {
        System.out.println("working in " + Thread.currentThread().getName());
    }
}
new Worker().start();

// 2. implement Runnable
new Thread(new Worker()).start();

// 3. a lambda, because Runnable has one method
new Thread(() -> System.out.println("working")).start();
```

start() and **run()** are not the same. **start()** asks the JVM for a new thread. **run()** is an ordinary method call on the current one:

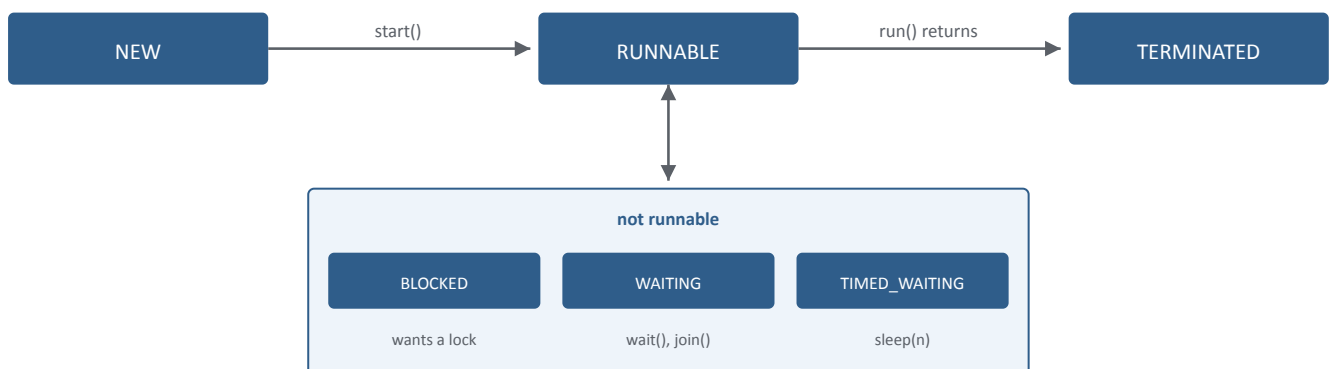
```
Thread thread = new Thread(() -> System.out.println(
    Thread.currentThread().getName()));

thread.start(); // prints Thread-0
thread.run();   // prints main
```

Extending **Thread** also uses the one inheritance a class has, for no benefit. Implementing **Runnable**, or passing a lambda, leaves the class free.

3. The thread lifecycle

Thread.getState() returns one of six values.



All three lower states return to **RUNNABLE**. Nothing goes back to **NEW**, and nothing leaves **TERMINATED**.

There is no RUNNING state. `RUNNABLE` covers both a thread the scheduler is running and one waiting its turn, because the JVM cannot see the difference. The distinction between the three lower states is what the thread is waiting for:

State	Waiting for
<code>BLOCKED</code>	A monitor lock held by another thread
<code>WAITING</code>	Another thread to act, through <code>wait()</code> , <code>join()</code> or <code>park()</code>
<code>TIMED_WAITING</code>	The same, or simply a period of time

A thread cannot be restarted. Calling `start()` on a terminated thread throws `IllegalThreadStateException`.

4. The race condition

```
class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        Thread a = new Thread(() -> { for (int i = 0; i < 10000; i++)
counter.increment(); });
        Thread b = new Thread(() -> { for (int i = 0; i < 10000; i++)
counter.increment(); });

        a.start();
        b.start();
        a.join();
        b.join();

        System.out.println(counter.count);
    }
}
```

Output:

```
17423
```

Not 20000, and a different number on the next run.

`count++` looks like one step and is three: read the value, add one, write it back. Two threads can read the same value, both add one to it, and both write the same result. One increment disappears.

```
thread A reads 41
thread B reads 41
thread A writes 42
thread B writes 42      one increment lost
```

A race condition is a bug that depends on timing. It passes every test on a laptop and fails once an hour in production, which is why the fix belongs in the design rather than in a retry.

5. synchronized

`synchronized` allows one thread at a time to hold an object's monitor lock.

```
class Counter {
    private int count = 0;

    synchronized void increment() {
        count++;
    }

    synchronized int get() {
        return count;
    }
}
```

The same program now prints:

```
20000
```

Every time.

Form	The lock it takes
<code>synchronized</code> instance method	The object, <code>this</code>
<code>synchronized</code> static method	The <code>Class</code> object
<code>synchronized (obj) { }</code> block	Whatever <code>obj</code> is

A block is usually better than a method, because it holds the lock for less time:

```

void record(Order order) {
    Report report = buildReport(order);    // slow, and needs no lock

    synchronized (this) {
        totals.add(report.total());        // the only part that shares state
    }
}

```

The getter needs it too. Locking only the writer leaves a reader free to see a half-updated value, or an old one it has cached.

Lock on a private field, not on `this`. Any code holding a reference to the object can synchronize on it as well, and then two unrelated pieces of code are competing for one lock:

```

private final Object lock = new Object();

```

6. volatile and atomic types

`volatile` solves visibility, not atomicity. Without it a thread may keep a field in a register or a cache and never see another thread's write:

```

private volatile boolean running = true;

public void stop() {
    running = false;    // every thread sees this
}

public void run() {
    while (running) { } // without volatile, this can loop forever
}

```

`volatile` does not make `count++` safe. The write is visible, and the three steps are still three steps. For a counter, an atomic type does both:

```

private final AtomicInteger count = new AtomicInteger();

void increment() {
    count.incrementAndGet();
}

```

That runs without a lock, using a compare-and-set instruction the processor provides.

Need	Use
One thread writes a flag, others read it	<code>volatile</code>
Several threads change one number	<code>AtomicInteger</code> , <code>AtomicLong</code>
Several threads swap one reference	<code>AtomicReference</code>
Several statements must happen together	<code>synchronized</code> or a lock

The atomic classes cover one variable. Two fields that must change together still need a lock.

7. Locks

`ReentrantLock` does what `synchronized` does, and adds what `synchronized` cannot.

```
private final ReentrantLock lock = new ReentrantLock();

void transfer(Account to, BigDecimal amount) {
    lock.lock();
    try {
        // the critical section
    } finally {
        lock.unlock();    // never outside a finally
    }
}
```

Feature	<code>synchronized</code>	<code>ReentrantLock</code>
Releasing the lock	Automatic, at the end of the block	Our job, in a <code>finally</code>
Trying without blocking	No	<code>tryLock()</code>
Giving up after a wait	No	<code>tryLock(2, SECONDS)</code>
Interruptible	No	<code>lockInterruptibly()</code>
Fairness	No	Optional, at a cost

`ReadWriteLock` is the other one worth knowing. Many readers may hold it at once, and a writer excludes everyone, which suits data read far more often than it changes.

`synchronized` remains the default, because the release cannot be forgotten. Use a lock when a feature in that table is needed.

8. Deadlock

Two threads each holding what the other wants.

```
// thread A
synchronized (first) {
    synchronized (second) { }
}

// thread B
synchronized (second) {
    synchronized (first) { }
}
```

If A takes `first` and B takes `second`, neither can continue and neither will give up. The JVM does not detect this, and the program hangs.

Four conditions have to hold together, and removing any one prevents it:

Condition	Removing it
Mutual exclusion	Use an immutable object or an atomic type
Hold and wait	Take every lock at once, or none
No preemption	Use <code>tryLock</code> with a timeout and release what we hold
Circular wait	Always take locks in the same order

The last one is the practical fix. Ordering locks by something stable, such as an account id, means two transfers between the same pair can never take them in opposite order.

Two relatives: **livelock**, where threads keep responding to each other and make no progress, and **starvation**, where a thread never gets the lock because others keep taking it.

9. Executors and thread pools

Creating a thread per task stops working quickly. A thread costs about a megabyte of stack, and thousands of them spend their time on context switches rather than work.

```
ExecutorService pool = Executors.newFixedThreadPool(4);

for (int i = 0; i < 100; i++) {
    int task = i;
    pool.submit(() -> System.out.println("task " + task));
}

pool.shutdown();
pool.awaitTermination(10, TimeUnit.SECONDS);
```

A hundred tasks, four threads, each picking up the next one as it finishes.

Factory	What it gives
<code>newFixedThreadPool(n)</code>	Exactly <code>n</code> threads, with an unbounded queue
<code>newCachedThreadPool()</code>	A new thread per task, reusing idle ones
<code>newSingleThreadExecutor()</code>	One thread, so tasks run in order
<code>newScheduledThreadPool(n)</code>	Tasks that run later or repeatedly
<code>newVirtualThreadPerTaskExecutor()</code>	A virtual thread per task, Java 21

Prefer `new ThreadPoolExecutor(...)` in production. The factory methods hide a choice that matters: `newFixedThreadPool` queues without limit, so a backlog becomes memory pressure instead of a rejected task, and the failure arrives as `OutOfMemoryError` rather than something a caller can handle.

`shutdown()` stops accepting work and lets running tasks finish. `shutdownNow()` interrupts them.

10. Future and CompletableFuture

`submit` returns a `Future`, which is a handle on a result that does not exist yet.

```
Future<BigDecimal> total = pool.submit(() -> calculateTotal(orderId));

BigDecimal value = total.get();    // blocks until it is ready
```

`Future.get()` blocks, which reintroduces the problem the pool was solving. `CompletableFuture` composes instead:

```
CompletableFuture<Employee> employee =
    CompletableFuture.supplyAsync(() -> repository.findById(id), pool);

CompletableFuture<BigDecimal> tax =
    CompletableFuture.supplyAsync(() -> taxService.rateFor(id), pool);

employee.thenCombine(tax, (e, rate) -> e.getSalary().multiply(rate))
    .thenAccept(System.out::println)
    .exceptionally(error -> {
        log.error("failed", error);
        return null;
    });
```

Both calls run at the same time, and the result is assembled when both finish.

Method	What it does
<code>supplyAsync</code>	Runs something that returns a value

Method	What it does
<code>thenApply</code>	Transforms the result
<code>thenCompose</code>	Chains another future, flattening it
<code>thenCombine</code>	Waits for two and merges them
<code>allOf</code> , <code>anyOf</code>	Waits for all, or the first
<code>exceptionally</code> , <code>handle</code>	Recovers from a failure

Always pass an executor. Without one these run on the common `ForkJoinPool`, which is shared with parallel streams and sized for processors rather than for waiting.

11. Concurrent collections

`ArrayList` and `HashMap` are not safe for concurrent use. A `HashMap` written by two threads can produce a corrupted structure.

Instead of	Use
<code>HashMap</code>	<code>ConcurrentHashMap</code>
<code>ArrayList</code>	<code>CopyOnWriteArrayList</code> , for many reads and few writes
A hand-built queue	<code>BlockingQueue</code> , <code>ConcurrentLinkedQueue</code>
<code>Collections.synchronizedMap</code>	<code>ConcurrentHashMap</code> , which locks less

A thread-safe collection does not make a sequence of calls safe:

```
if (!map.containsKey(key)) {    // two threads can both pass this
    map.put(key, value);
}

map.putIfAbsent(key, value);    // one call, and safe
```

Each method is atomic. Two of them together are not, which is why `ConcurrentHashMap` supplies `putIfAbsent` , `computeIfAbsent` and `merge` .

12. Virtual threads

Java 21 added threads the JVM schedules itself, rather than the operating system. They are cheap enough to create one per task.

```
try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) {
        pool.submit(() -> callAnotherService());
    }
}
```

A hundred thousand platform threads is impossible. A hundred thousand virtual threads is ordinary, because a virtual thread that blocks on input or output is unmounted from its carrier thread, which then runs something else.

	Platform thread	Virtual thread
Backed by	An operating system thread	A JVM object on a carrier thread
Cost to create	About a megabyte of stack	A few hundred bytes
How many	Thousands	Millions
Good for	Work that uses the processor	Work that waits

They help waiting, not computing. A task that spends its time on the processor gains nothing, because the work still needs a real core. The gain is in a service that spends its time waiting for a database or another service, which is most services.

Pooling them is also unnecessary. The pool existed because threads were expensive, and they are no longer expensive.

The pieces together

Piece	What it solves
<code>synchronized</code>	One thread at a time through a section
<code>volatile</code>	One thread's write becomes visible to the others
<code>AtomicInteger</code>	A counter without a lock
<code>ReentrantLock</code>	A lock that can time out or be interrupted
Lock ordering	Deadlock
<code>ExecutorService</code>	Reusing threads instead of creating them
<code>CompletableFuture</code>	Combining results without blocking
<code>ConcurrentHashMap</code>	A map several threads can use
Virtual threads	One thread per task, when tasks wait

Consider an order service under load. Each request already runs on its own thread, so most code needs nothing. A cache of exchange rates is a `ConcurrentHashMap` filled with `computeIfAbsent`. A report that calls three services uses `CompletableFuture` so the three calls overlap. A running total is an `AtomicLong`. The one `synchronized` block in the codebase guards two fields that have to change together, and it locks a private object rather than `this`.

Take one field that two threads can reach and decide which it needs: nothing, because it never changes; `volatile`, because one thread writes it; an atomic type, because it is one number; or a lock, because it is more than one field.